

Visualización interactiva y detección de anomalías en micromovilidad eléctrica mediante Shiny

David Villegas Ceballos , Olga Cecilia Usuga Manco

Palabras clave: Shiny - Análisis de datos - Micromovilidad eléctrica - Grande volúmenes de datos

Introducción

La topografía andina, presente en ciudades como Medellín, somete a los vehículos de micromovilidad eléctrica a niveles de exigencia variables que impactan directamente su rendimiento. Para analizar este fenómeno, se requiere la instrumentación de los vehículos con sensores que registren su comportamiento en condiciones de tráfico real. Estos dispositivos capturan de forma simultánea múltiples variables operativas (corrientes, voltajes y temperaturas) segundo a segundo, generando volúmenes masivos de información que superan los 78.000 registros por cada experimento. Sin embargo, procesar y explorar esta gran cantidad de datos de forma tradicional mediante scripts estadísticos estáticos resulta ineficiente. Identificar picos críticos, detectar correlaciones anómalas y diagnosticar fallas exige la reescritura constante de código para cada iteración, lo que genera cuellos de botella y retrasa la toma de decisiones técnicas. Ante este desafío analítico, se propone el desarrollo de un aplicativo interactivo en Shiny diseñado para la exploración ágil de datos operativos. Esta plataforma transforma registros crudos en interfaces visuales y reactivas, permitiendo a investigadores y tomadores de decisiones cruzar variables y evaluar el rendimiento de los sistemas de micromovilidad sin necesidad de interactuar directamente con la programación subyacente.

Metodología

Para la construcción del aplicativo Shiny se diseñó un flujo de trabajo computacional estructurado en cinco fases, abarcando desde la captura de datos hasta el despliegue de la interfaz interactiva:

Fase 1. Obtención de datos: La información base proviene de la instrumentación de una bicicleta eléctrica mediante sensores acoplados a su batería y a las tres líneas de su motor trifásico, lo que genera cuatro nodos de lectura simultánea. En estos puntos se registraron variables de temperatura, voltaje, corriente y potencia a una frecuencia de 1 Hz. El protocolo experimental consistió en recorridos donde se agotaba la batería desde su carga máxima bajo tres escenarios de exigencia física. El primero fue la ruta Metroplús, un trayecto urbano sin pendientes pronunciadas pero con paradas constantes debido a los semáforos. El segundo fue la ruta Palmas, caracterizada por su alta exigencia debido a las pendientes prolongadas. Finalmente, el tercer escenario consistió en simulaciones controladas en laboratorio. Esta diversidad en las condiciones de manejo generó un volumen masivo de datos con comportamientos muy variados, los cuales el aplicativo debe procesar y organizar.

Fase 2. Limpieza: Para que los datos brutos se puedan ver bien en las gráficas, se usó el ecosistema Tidyverse. Con el paquete readr se cargó la información y con dplyr se limpiaron los registros que tenían errores de lectura de los sensores. Después, los datos se organizaron por fecha, lugar y número de experimento. También se ajustó la línea de tiempo para que fuera consistente en todos los casos evaluados. Este orden previo es lo que permite que el Shiny funcione de forma fluida y reactiva.

Fase 3. Detección de anomalías: Los datos procesados ingresan al aplicativo con una clasificación previa. Cada registro cuenta con un puntaje de riesgo y una etiqueta que diferencia los comportamientos normales de las anomalías detectadas. Esto permite que el Shiny identifique y resalte visualmente los puntos críticos en las gráficas de manera inmediata.

Fase 4. Arquitectura del aplicativo: El desarrollo de la plataforma se realizó en Shiny, dividiendo el diseño en la interfaz visual (UI) y la lógica de procesamiento (Server). Para manejar eficientemente el alto volumen

de datos sin saturar la memoria, se implementaron funciones reactivas. Esto permite aislar y cargar únicamente los datos del experimento que el usuario selecciona. Adicionalmente, se utilizaron funciones de control como `req()` y `validate()` para asegurar que las gráficas solo se generen cuando la información requerida esté completamente disponible, evitando que el sistema colapse o muestre errores en pantalla al cambiar de ruta.

Fase 5. Visualización interactiva: La presentación final de los resultados se construyó integrando dos librerías principales. Por un lado, se utilizó el paquete `plotly` para renderizar las series de tiempo dinámicas, lo que le permite al usuario hacer acercamientos simultáneos en los picos de voltaje o temperatura para detallar el instante exacto de una anomalía. Por otro lado, se implementó `ggplot2` para generar matrices de correlación que resumen visualmente la relación entre las diferentes variables, facilitando el diagnóstico del estado del vehículo en cada escenario.

Resultados